

Dataframes and SparkSQL

Sure! Let's dive into the concept of **SparkSQL** and **DataFrames** in simple terms.

SparkSQL and DataFrames: A Beginner's Guide

SparkSQL is like a powerful tool that helps you work with structured data, similar to how you might use a spreadsheet to organize and analyze information. Imagine you have a big box of LEGO bricks (your data) and you want to build something specific (like a report). SparkSQL allows you to use a special set of instructions (SQL queries) to find and arrange those bricks in the way you want. It can handle data in various programming languages like Python, Java, and R, making it very flexible.

Now, think of a DataFrame as a well-organized table in a database or a spreadsheet. Each column in this table has a name, and the rows contain the actual data. For example, if you have a DataFrame of people, one column might be "Name," and another could be "Age." DataFrames are great because they can grow from small amounts of data to huge datasets, and they come with built-in tools to make your queries faster and more efficient.

Spark SQL

- Is a Spark module for structured data processing
- Used to query structured data inside Spark programs, using either SQL or a familiar DataFrame API
- Usable in Java, Scala, Python and R
- Runs SQL queries over imported data and existing RDDs independently of API or programming language

- Spark SQL is a Spark module for structured data processing. You can interact with Spark SQL using SQL queries and the DataFrame API. Spark SQL supports Java, Scala, Python, and R APIs. Spark SQL uses the same execution engine to compute the result independently of which API or language you are using for the computation. Developers can use the API that provides the most natural way to express a given transformation.

Spark SQL example

Spark SQL query using Python

```
results = spark.sql(  
    "SELECT * FROM people")  
names = results.map(lambda p:  
    p.name)
```

- Here is an example of a Spark SQL query using Python. The query select star from people statement is the SQL query run using Spark SQL. The entity "people" was registered as a table view before this command.

Spark SQL - Benefits

- Includes a cost-based optimizer, columnar storage, and code generation to make queries fast
 - Scales to thousands of nodes and multi-hour queries using the Spark engine, which provides full mid-query fault tolerance
 - Provides a programming abstraction called DataFrames and can also act as a distributed SQL query engine
-
- Unlike the basic Spark RDD API, Spark SQL includes a cost-based optimizer, columnar storage, and code generation to perform optimizations that provide Spark with additional information about the structure of both the data and the computation in process.

DataFrames

- Distributed collection of data organized into named columns
 - Conceptually equivalent to a table in a relational database or a data frame in R/Python, but with richer optimizations
 - Built on top of the RDD API
 - Uses RDDs
 - Performs relational queries
-
- So, what is a DataFrame? A DataFrame is a collection of data organized into named columns. DataFrames are conceptually equivalent to a table in a

relational database and similar to a data frame in R/Python, but with richer optimizations. DataFrames are built on top of the Spark SQL RDD API. DataFrames use RDDs to perform relational queries.

DataFrame

Python code snippet to read from a JSON file and create a simple DataFrame.

```
df = spark.read.json("people.json")
df.show()
df.printSchema()

# Register the DataFrame as a SQL temporary view
df.createTempView("people")
```

- Here is a simple code snippet in Python to read from a JSON file and create a simple DataFrame. The last step of registering the DataFrame runs SQL queries on this data using Spark SQL.

DataFrame example

Input JSON file

```
{"name": "Michael"}  
{"name": "Andy",  
 "age": 30}  
{"name": "Justin",  
 "age": 19}
```

Created DataFrame

```
+----+-----+  
| age| name |  
+----+-----+  
| null| Michael|  
| 30| Andy |  
| 19| Justin|  
+----+-----+
```

- Here you see the input data file in JSON format and the resulting DataFrame.

DataFrame Benefits

- Ability to scale from kilobytes of data on a single laptop to petabytes on a large cluster
 - Support for a wide array of data formats and storage systems
 - State-of-the-art optimization and code generation through the Spark SQL Catalyst optimizer
 - Seamless integration with all big data tooling and infrastructure via Spark
 - APIs for Python, Java, Scala, and R, which is in development via Spark R
-
- DataFrames offer many benefits: DataFrames are highly scalable—from a few kilobytes on a single laptop to petabytes on a large cluster of machines. DataFrames support a wide array of data formats and storage systems. DataFrames provide optimization and code generation through a Catalyst

optimizer. Last, DataFrames are developer-friendly, offering integration with most big data tooling via Spark and APIs for Python, Java, Scala, and R

DataFrame + Spark SQL

SQL Query	Result
<pre>spark.sql("SELECT name FROM people").show()</pre>	<pre>+-----+ name +-----+ Michael Andy Justin +-----+</pre>
DataFrame Python API	
<pre>df.select("name").show() df.select(df["name"]).show()</pre>	

- These code snippets show how to run the same SQL query and the result of those queries using Spark SQL. All three queries show here achieve the same result of showing the names column from our DataFrame. The first query is a more traditional SQL query, while the other two code examples use the DataFrame Python API to perform the same task.

DataFrame + Spark SQL

SQL Query

```
spark.sql("SELECT age, name  
FROM people WHERE age >  
21").show()
```

DataFrame Python API

```
df.filter(df["age"]>21).show()
```

Result

```
+---+-----+  
|age|name|  
+---+-----+  
| 30|Andy|  
+---+-----+
```

- Similarly, here are two different ways to query the entries in the DataFrame to locate people who are above the age of 21. The first code example is a SQL query, while the second code example uses the DataFrame API.

Summary

In this video, you learned that:

- Spark SQL is a Spark module for structured data processing
- Spark SQL provides a programming abstraction called DataFrames and can also act as a distributed SQL query engine
- DataFrames are conceptually equivalent to a table in a relational database or a data frame in R/Python, but with richer optimizations